(11) FUNCTIONS & METHODS

Date: / / Page no:

Introduction to function/methods :-

⟹ BLOCK of code / reusable code.

Syntax :-

```
return-type name (){

        // body
        return statement;
    }
```

Code :-

```
public class Vj{
    public static void ~~main~~ Hello (String [] args){

        Sopln ("Hello world");
        Sopln ("Hello world");
        // return;
    }
    public static void main (String [] args){

        printHelloworld ();    //function call
    }
}
```

→ output :- Hello world
Hello world

Note: we can make multiple functions in single class.

Date: / / Page no:

Syntax with Parameters:-

```
returnType name (type Param1, type Param2){

        //body
        return statement;
}
```

```
import java.util.*;

public class functions{
    public static int calculateSum (int num1, int num2){

        int sum = num1 + num2;
        return sum;
    }
    public static void main (String [] a){

        Scanner sc = new Scanner (System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int sum = CalculateSum (a, b); //argument or actual Param
        Sopln ("Sum is :" + sum);
    }
}
```

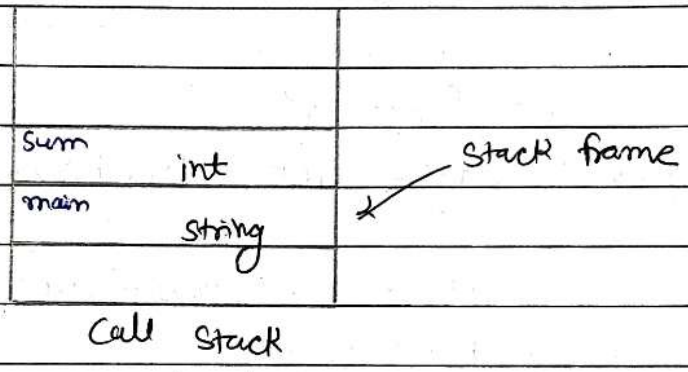
→ Output :- 7
2
Sum is : 9

Parameters Vs Arguments

Defination
- Parameters : formal Parameters or Parameter

Call
- Arguments :- Actual Parameters or Arguments

What happens in memory ?



Call by value :

⇨ Program to swap 2 numbers :-

```
import java.util.*;

public class functions {
        public static void swap(int a, int b) {

                int temp = a;
                a = b;
                b = temp;
                return;
        }
```

Java always call by value : In java all functions always work on call by value. It means it sends copy of variable.

Date: / / Page no:

```
public static void main (String a[]){

        int a = 5;
        int b = 10;

        swap (a,b);

        Sopln("a = "+a);
        Sopln("b = "+b);
    }
}
```

→ output: a = 10
b = 5

Find Product of a & b.

a = 3 , b = 5

```
import java.util.*;

public class function{
    public static int multiply (int a, int b) {

        int product = a*b;
        return product;
    }
    public static void main (String [] a){
```

```
        int prod;

        prod = multiply (3,5);
        Sop ln (" Product is :"+ prod);
    }
}
```

→ Output : Product is : 15

☐ Factorial of number (n).

$n!$ $(n=4)$

$n! = n \times (n-1) \times (n-2) \times (n-3) \times --- \times 1$

or

$n! = 1 \times 2 \times 3 \times ------ \times n$

Sp
```
import java.util.*;

public class function {

    public static int factorial (int n) {

        int f = 1;

        for (int i = 1; i<=n; i++) {

            f = f * i;
        }
```

```
        return f;
    }
    public static void main (String [] a){
        Sop.ln("Factorial is:"+ (factorial (5));
    }
}
```

→ Output :- Factorial is : 120

□ Binomial Coefficient $(n=5, r=2)$

$$^{n}C_{r} = \frac{n!}{r!(n-r)!}$$

```
int binCoeff (int n, int r){
    int n-fact = factorial (n);
    int r-fact = factorial (r);
    int nmr-fact = factorial (n-r);
```

```
import java.util.*;
public class function{
    public static int factorial (int n){
        int f = 1;
```

```
        for(int i=1; i<=n; i++){
            f = f * i;
        }
        return f;
    }
    public static int Bincoeff (int n, int r) {
        int n_fact = factorial(n);
        int r_fact = factorial (r);
        int nmr_fact = factorial (n-r);

        int Binco = n_fact /(r_fact * nmr_fact);

        return Binco;
    }

    public static void main (String [] a){
        Sopln( Bincoeff (5,2));
    }
```

→ Output : 10

Inbuilt vs User Defined Methods:-

Methods

User defined
- factorial
- sum
- --- etc

Inbuilt functions
- Math → Pow, Sqrt, max
- Sc.nextInt()

Function overloading :-

Multiple functions with the same name but different parameters.

Calculator

```
multiply (int a, int b)      // integer
multiply (float a, float b)  // float value
multiply (double a, double b) // double value
multiply (int a, int b, int c) // three value
```

Note :- function /method overloading is depends on parameter not on return type.

```
int sum (- -)   X
float sum (- -) X
```

Date: / / Page no:

Function overloading using parameters:

f1: Sum of 2 no :- int sum (int a, int b) {
return a+b;
}

f2: Sum of 3 no :- int sum(int a, int b, int c) {
return a+b+c;
}

Code

```
import java.util.*;
public class functions {
    public static int sum(int a, int b){
        return a+b;
    }
    public static int sum(int a, int b, int c){
    }

    public static void main (String [] a){
        Sopln ( sum(2,3));
        Sopln ( sum (2,3,4));
    }
}
```

→ Output: 5
9

Function / Method overloading using data type :-

∴ add 2 int value :- int sum (int a, int b) {
return a+b;
}

∴ add 2 float value :- float sum (~~int~~ float a, ~~int~~ float b) {
return a + b;
}

code :-

```
public class functions{

    public static int sum (int a, int b) {
            return a+b;
    }
    public static float sum.(float a, float b) {
            return a+b;
    }

    public static void main (String [] a){

            Sopln (sum (3,6));
            Sopln (sum( 4.5f , 3.4f));
    }
```

→ Output :- 9
7.9

* optimized way : at (n-1) use Math.sqrt(n)

Date: / / Page no:

- Check if a number is prime or not using method :-

```
public class functions{

    public static boolean isprime (int n){

        if (n==2){
            return true;
        }

        boolean isprime = true;
        for(int i=2; i< n-1; i++){
                *
            if (n%i==0){

                isprime = false;
            }
        }

        return isprime;
    }

    public static void main (String [] args){

        Sopln (isprime (5));
    }
```

→ Output :- 120.

- Print all prime in a Range (n=10) :-

2 to 10 = 2, 3, 5, 7

Date: / / Page no:

code

```
public class functions {

    public static void primeInRange (int n){
            // Same code as in previous question
        for(int i=2 ; i<=n ; i++) {
            if ( isprime (i)){

                Sop (i+" ");
            }
        }
        Sopln();
    }
    public static void main (String [] a){
        primeInRange (20);
    }
}
```

→ Output:- 2, 3 5 11 13 17 19

Binary to Decimal conversion (n = 101)

Math → 0 to 9

$\underbrace{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}_{\text{10 digit} = \text{Dec}}$

How to Solve any Problem:-
(i) Paper - Pen
(ii) logic
(iii) Code

Date: / / Page no:

$(0)_{10} = (0)_2$
$(1)_{10} = (01)_2$
$(2)_{10} = (10)_2$
$(3)_{10} = (11)_2$
$(4)_{10} = (100)_2$
$(5)_{10} = (101)_2$
$(6)_{10} = (110)_2$
$(7)_{10} = (111)_2$
$(8)_{10} = (1000)_2$

$n = 1 \quad 0 \quad 1$

$\downarrow * \quad \downarrow * \quad \downarrow *$

$= 2^2 + 2^1 + 2^0$

$= 4(1) + 2(0) + 1(1)$

$= 4 + 0 + 1 = 5$

$(101)_2 = (5)_{10}$

decimal = 0

① dec = dec + lastdigit * 2^{pow} = $0 \times 1 \times (2^0)$

LD = 1

② dec = dec + 1 * 2^1

$= 1 + 1(2) = 3$

Code

```
public class function{

    public static void binToDec (int binNum) {

        int mynum = binNum;
        int pow = 0;
        int decimal = 0;

        while (binNum > 0) {

            int lastdigit = binNum % 10;
```

```
        decimal = decimal + (lastdigit * (int) Math.pow(2, pow));

        pow++;
        binNum = binNum/10;
    }
    Sopln("decimal of "+ mynum + "is" + decimal);
}
public static void main (String [] args){

        binToDec (101);
    }
}
```

→ Output: decimal of 101 is 5.

■ Convert from Decimal to Binary (n = 7)

2	7	→ rem 1
2	3	1
2	1	1
	0	

2	5	1
2	2	0
2	1	1
	0	

$7_{10} = (111)_2$ $5_{10} = (101)_2$

2	11	1
2	5	1
2	2	0
2	1	1
	0	

$(11)_{10} = (1101)_2$

```
while (n > 0) {
① divide by 2 → remainder
② bin = bin + rem * 10^pow
    pow++;
}
```

Date: / / Page no:

Code:-

```
public class function {

    public static void decToBin (int n){

        int mynum = n;
        int pow = 0;
        int binNum = 0;

        while (n>0) {
            int rem = n % 2;
            binNum = binNum + (rem * (int) Math.pow (10, Pow))

            Pow ++;
            n = n/2;
        }
        Sopln("Binary of"+myNum + "is" +binNum);
    }
    public Static void main (String [] a){

        decToBin (7);
    }
}
```

→ Output:- Binary of 7 is 111

Block Scope:-

Date: / / Page no:

→ public static void main(String [] a){

```
int p = 10;    // for full function
{
    int s = 45;    // Block of scope
    Sopln (P);
}
// Sopln (S);
Sopln (p);
}
```

- Class scope :- Acheived using access modifier.

□ Scope :-

Method scope :-

```
function {
    Variable
    - - - -
    - - - -
}
```

#> PSVM (s [] a){

```
Sopln (s)      // Not Valid
int s = 45;    // Delaration
Sopln (s);     // Valid
}
```

Methods / Function Questions:-

Date: / / Page no:

Q.1 Write a Java method to compute the average of three numbers.

```
import java.util.Scanner;

public class funQue {
    public static void main (String [] args){

        Scanner sc = new Scanner (system.in);

        Sopln(" Enter the first no:");
        double x = sc.nextDouble();

        Sopln("Enter the Second no:");
        double y = sc.nextDouble();

        Sopln("Enter the third no:");
        double z = sc.nextDouble();

        Sop("The avg. value is:" + avg(x,y,z)+"\n");
    }

    public static ~~void~~ double avg(double x, double y,
                                      double z){
        return (x+y+z)/3;
    }
}
```

→ Output:- Enter the first no:
2.66

Enter the second no:
3.5

Enter the Third no:
4:00

The avg. value is 3.8666666 - - - - .

Q2 Write a method named isEven that accepts an int argument. The method should return true if the argument is even or false otherwise. Also write a program to test your method

→
```
import java.util.*;

public class Solution{
    public static boolean isEven(int number){

        if(number %2 ==0){
            return true;
        }
        else{
            return false;
        }
    }
    public static void main(String [] a){

        Scanner sc = new Scanner(System.in);
        int num ;

        System.out.print("Enter an integer: ");
```

```
        num = sc.nextInt();

        if( isEven (num)) {
                Sopln("Number is even");
        }
        else{
                Sopln("Number is odd");
        }
    }
}
```

→ Output :- Enter an integer :- 10
Number is even.

Q.3 Write a Java program to check if a number is a palindrome in Java? (121 is a palindrome, 321 is not)?

- Palindrome : A number is called a palindrome if the number is equal to the reverse of a number e.g, 121 is a palindrome because the reverse of 121 is 121 itself.
On the other hand, 321 is not a palindrome because the reverse of 321 is 123, which is not equal to 321.

```
import java.util.Scanner;

public class Solution{
```

Date: / / Page no:

```
public static void main (String [] a){

        Sopln("Please Enter a no:");
        Scanner sc = new Scanner (System.in);
        int palindrome = sc.nextInt();

        if (isPalindrome (palindrome)) {
                Sopln("Number:"+palindrome+"is a palindrome");
        } else {
                Sopln("Number:+palindrome+ "is not a palindrome");
        }
}
public static boolean isPalindrome(int number){

        int palindrome = number; //copy no. into variable
        int reverse = 0;

        while (palindrome != 0){

                int remainder = palindrome % 10;
                reverse = reverse * 10 + remainder;
                palindrome = Palindrome / 10;
        }
//if original and the reverse of number is equal
//mans number is palindrome.

        if (number == reverse){
                return true;
        }.
        return false;
    }
}
```

Date: / / Page no:

→ Output :- Please enter a number : 121

Number 121 is a palindrome.

Q.4 Read the article

(a) Math.min () : Returns the smaller of 2 number

⇒ Math.min (a, b)

(b) Math.max() : Returns the larger of 2 no.

⇒ Math.max (a, b)

(c) Math.sqrt () : Returns the square root of a number

⇒ Math.sqrt (number)

(d) Math.pow() : Return the value of a number raised to a power

⇒ Math.pow (base, Exponent)

(e) Math.avg () : ~~was~~ Right Now does not exist in Java

(f) Math.abs () : Return the absolute (Positive) value of a no

⇒ Math.abs (number)

Date: / / Page no:

Q.5 write a Java method to compute the sum of the digit in an integer.

Approach :-

(a) Take a Variable sum = 0
(b) Find the ~~variable~~ ~~sum~~ last digit of the number.
(c) Add it the sum
(d) Repeat a & b until the no. becomes (0).

```
import java.util.Scanner;

public class Solution {

    public static int sumOfDigit (int n) {

        int sum = 0;

        while (n > 0) {

            int lastDigit = n % 10;
            sum += lastDigit;
            n /= 10;
        }

        return sum;
    }

    public static void main (String [] a) {
```

Date: / / Page no:

```
Scanner sc = new Scanner (System.in);
Sop("Input an integer :");
int digit = sc.nextInt();
Sopln("The sum is "+ sumOfDigit(digits));
}
}
```

→ Output:- Input an integer: 12345

The sum is 15.